The most closely related work to ours is LISA [34], which fine-tunes a pretrained language model on a large Isabelle/HOL proof corpus, and uses it inside of a search procedure to predict proof steps. GPT-f [66] likewise combines a generative language model with proof search to target the Metamath proof language. A Monte-Carlo tree search approach outperforms GPT-f in Lean [42].

TacticZero [91] learns not just tactics but also proof search strategies for end-to-end proof synthesis, rather than relying on a single fixed proof search strategy like other neural theorem proving approaches. The approach works by way of deep reinforcement learning, and improves over the previous state of the art on a benchmark for the HOL4 theorem prover.

A related problem to neural theorem proving is *autoformalization*: the automatic translation of natural language specifications and proofs into formal, machine-checkable specifications and proofs. LLMs have shown promise for autoformalization of specifications, and automatically generated proofs of the resulting autoformalized specifications have been used to improve a neural theorem prover on a widely used benchmark suite in Isabelle/HOL [92]. ProofNet [3] introduces a dataset and benchmark suite for autoformalization in Lean, based on undergraduate mathematics, and shows preliminary promising results autoformalizing proofs on that benchmark using Codex [12] with few-short learning. Autoformalization of both theorems and proofs in Coq shows promise on a small preliminary benchmark suite [16]. Autoformalization for specification logics in verification is also promising [26].

The Draft, Sketch, and Prove method (DSP) [35] presents a hybrid between theorem proving and autoformalization, which, similar to our approach, makes use of LLMs for theorem proving. It provides informal proofs as drafts for the LLM to translate into a formal proof sketch, which is then proven via Sledgehammer. In contrast, we use fine-tuning for LLMs, do not make use of Sledgehammer, and do not rely on the availability of natural language proofs.

Pretrained language models can be used to answer natural-language mathematics questions [60]. Large language models, such as Minerva [48] and PaLM [14], have been evaluated on natural language mathematics benchmarks, such as GSM8k [15] and MATH [29]. The ProofNet [3] benchmark suite mentioned above includes informal proofs alongside formal proofs as a benchmark.

We introduce the proof repair task, with error messages. This is a new machine learning task for formal proofs. We show that solving this task improves neural theorem proving performance. Proof engineers perform proof repair constantly during formal proof development [72]. Automating this task first arose with the advent of symbolic tools for automatic proof repair in the Coq proof assistant [70], and has since made its way into tools for other proof systems [51]. Our work is among the first to explore proof repair in a machine learning context, and the first we are aware of to use error messages for a proof repair task, and to use repair to improve performance of proof synthesis.

There are numerous other tasks that machine learning tools for proofs consider that may either help users with proof development directly, or improve neural theorem proving performance themselves. For example, PaMpeR [55] predicts proof methods alongside explanations in Isabelle/HOL. ACL2(ml) [30] generates helper lemmas and suggests similar theorems in ACL2. Other popular tasks leveraging machine learning include premise selection and datatype alignment, and are described in more detail in QED at Large [71].

Our approach can help minimize human effort in formal verification by automatically synthesizing proofs for some theorems. Other tools that assist humans writing formal verification proofs can similarly save time, and can be complementary to our work for theorems Baldur cannot prove fully automatically. iCoq [8, 9], and its parallelized version PiCoq [63], find failing proof scripts in evolving projects by prioritizing proof scripts affected by a revision. iCoq tracks fine-grained dependencies between Coq definitions, propositions, and proof scripts to narrow down the potentially affected proof scripts. QuickChick [43], a random testing tool for Coq, searches for counterexamples to executable theorems, helping a programmer to become more confident that a theorem is correct. Roosterize [56, 58] can suggest names for lemmas, and language models can also help automatically format proofs [57], both improving readability and maintainability. Mutation analysis can identify weak specifications, when mutating definitions does not break their proofs [10, 32]. The mutation operators could, hypothetically, be applied in repair and in providing feedback for developers as to why a proof has broken.

The automated program repair field studies the task of taking a program with a bug, evidenced by one or more failing tests, and automatically producing a modified version of the program that passes all the tests [46]. Generate-and-validate repair techniques use search-based techniques or predefined templates to generate many syntactic candidate patches, validating them against the tests (e.g., GenProg [45], Prophet [49], AE [89], HDRepair [44], ErrDoc [83], JAID [11], Qlose [18], and Par [39], ssFix [93], CapGen [90], SimFix [37], Hercules [74], Recoder [102], among others). Techniques such as DeepFix [25] and ELIXIR [73] use learned models to predict erroneous program locations, as well as the patches. It is possible to learn how to repair errors together by learning how to create errors, which can increase the amount of available training data, but poses an additional challenge of learning to approximate making human-like errors [98]. Unfortunately, these automated program repair techniques often overfit to the available tests and produce patches that, while passing all the tests, fail to encode the developers' intent [54, 59, 68, 78]. Improving the quality of the resulting repairs can be done via improving fault localization strategies [2, 36, 41, 50, 53, 80, 94], patch generation algorithms (e.g., heuristic-based [37, 45, 49, 65, 83, 90], constraint-based [1, 24, 38, 52, 86], and learning-based [13, 25, 73]), and patch validation methodologies [82, 87, 95, 99, 100]. By contrast, in Baldur's domain of theorem proving, it is impossible to produce a proof that appears to prove the theorems, but actually fails to do so, because the theorem prover acts as an absolute oracle for the correctness of the proof. As a result, it may be more difficult to produce a proof in the first place, but if techniques in this domain do produce proofs, they are guaranteed to be correct.

6 CONTRIBUTIONS

This paper is the first to fine-tune large language models to generate entire proofs of theorems without the need for proof search or hammers. We demonstrate that this approach is more effective and more efficient than prior methods that use one-step-at-a-time

search-based generation, and that it is complementary to existing search-based and hammer-based approaches: Together, our Baldur and prior tools can fully automatically synthesize proofs for 65.7% of the theorems in a large Isabelle/HOL benchmark, establishing a new state of the art. We further demonstrate that generate-and-repair improves proof synthesis when the language model is given access to the error messages produced by erroneous proofs.

This work opens new avenues of research into (1) using LLMs to automate theorem proving and simplify formal verification of software properties, (2) repair approaches, both for proofs and, potentially, more traditional automated program repair tasks, and (3) the use of context (e.g., failed synthesis attempts and error messages) in proof generation. Our very encouraging results suggest a bright future for automated proof generation and repair using LLMs.

ACKNOWLEDGMENTS

We thank Stella Biderman, Ernest Davis, and others who provided feedback on an earlier draft of this paper. This work is supported by the Defense Advanced Research Projects Agency under grant no. DARPA HR0011-22-9-0063, and by the National Science Foundation under grant no. CCF-2210243.

REFERENCES

[1] Afsoon Afzal, Manish Motwani, Kathryn T. Stolee, Yuriy Brun, and Claire Le Goues. 2021. SOSRepair: Expressive Semantic Search for Real-World Program Repair. *TSE* 47, 10 (October 2021), 2162–2181. https://doi.org/10.1109/TSE.2019.2944914

[2] Fatmah Yousef Assiri and James M Bieman. 2017. Fault Localization for Automated Program Repair: Effectiveness, Performance, Repair Correctness. *Software Quality Journal* 25, 1 (2017), 171–199. https://doi.org/10.1007/s11219-016-9312-z

[3] Zhangir Azerbayev, Bartosz Piotrowski, and Jeremy Avigad. 2022. ProofNet: A benchmark for autoformalizing and formally proving undergraduate-level mathematics problems. In *Workshop MATH-AI: Toward Human-Level Mathematical Reasoning*. New Orleans, Louisiana, USA.

[4] Kshitij Bansal, Sarah M. Loos, Markus N. Rabe, Christian Szegedy, and Stewart Wilcox. 2019. HOList: An Environment for Machine Learning of Higher Order Logic Theorem Proving. In *Proceedings of the 36th International Conference on Machine Learning, ICML 2019, 9-15 June 2019, Long Beach, California, USA (Proceedings of Machine Learning Research, Vol. 97)*, Kamalika Chaudhuri and Ruslan Salakhutdinov (Eds.). PMLR, 454–463. http://proceedings.mlr.press/v97/bansal19a.html

[5] Lasse Blaauwbroek, Josef Urban, and Herman Geuvers. 2020. The Tactician. In *Intelligent Computer Mathematics*, Christoph Benzmüller and Bruce Miller (Eds.). Springer International Publishing, Cham, 271–277.

[6] Sidney Black, Stella Biderman, Eric Hallahan, Quentin Anthony, Leo Gao, Laurence Golding, Horace He, Connor Leahy, Kyle McDonell, Jason Phang, Michael Pieler, Usvsn Sai Prashanth, Shivanshu Purohit, Laria Reynolds, Jonathan Tow, Ben Wang, and Samuel Weinbach. 2022. GPT-NeoX-20B: An Open-Source Autoregressive Language Model. In *Proceedings of BigScience Episode #5 – Workshop on Challenges & Perspectives in Creating Large Language Models*. Association for Computational Linguistics, virtual+Dublin, 95–136. https://doi.org/10.18653/v1/2022.bigscience-1.9

[7] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. 2020. Language Models are Few-Shot Learners. In *NeurIPS*.

[8] Ahmet Celik, Karl Palmskog, and Milos Gligoric. 2017. ICoq: Regression proof selection for large-scale verification projects. In *IEEE/ACM International Conference on Automated Software Engineering (ASE)*. Urbana-Champaign, IL, USA, 171–182. https://doi.org/10.1109/ASE.2017.8115630

[9] Ahmet Celik, Karl Palmskog, and Milos Gligoric. 2018. A Regression Proof Selection Tool for Coq. In *International Conference on Software Engineering Demonstrations Track (ICSE DEMO)*. Gothenburg, Sweden, 117–120. https://doi.org/10.1145/3183440.3183493

[10] Ahmet Celik, Karl Palmskog, Marinela Parovic, Emilio Jesús Gallego Arias, and Milos Gligoric. 2019. Mutation Analysis for Coq. In *IEEE/ACM International Conference on Automated Software Engineering (ASE)*. San Diego, California, 539–551. https://doi.org/10.1109/ASE.2019.00057

[11] Liushan Chen, Yu Pei, and Carlo A. Furia. 2017. Contract-based program repair without the contracts. In *IEEE/ACM International Conference on Automated Software Engineering (ASE)*. Urbana, IL, USA, 637–647.

[12] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harrison Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, Dave Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William Hebgen Guss, Alex Nichol, Alex Paino, Nikolas Tezak, Jie Tang, Igor Babuschkin, Suchir Balaji, Shantanu Jain, William Saunders, Christopher Hesse, Andrew N. Carr, Jan Leike, Joshua Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, and Wojciech Zaremba. 2021. Evaluating Large Language Models Trained on Code. *CoRR* abs/2107.03374 (2021). arXiv:2107.03374 https://arxiv.org/abs/2107.03374

[13] Zimin Chen, Steve James Kommrusch, Michele Tufano, Louis-Noël Pouchet, Denys Poshyvanyk, and Martin Monperrus. 2019. Sequencer: Sequence-to-sequence learning for end-to-end program repair. *TSE* 47, 9 (2019), 1943–1959. https://doi.org/10.1109/TSE.2019.2940179

[14] Aakanksha Chowdhery, Sharan Narang, Jacob Devlin, Maarten Bosma, Gaurav Mishra, Adam Roberts, Paul Barham, Hyung Won Chung, Charles Sutton, Sebastian Gehrmann, Parker Schuh, Kensen Shi, Sasha Tsvyashchenko, Joshua Maynez, Abhishek Rao, Parker Barnes, Yi Tay, Noam Shazeer, Vinodkumar Prabhakaran, Emily Reif, Nan Du, Ben Hutchinson, Reiner Pope, James Bradbury, Jacob Austin, Michael Isard, Guy Gur-Ari, Pengcheng Yin, Toju Duke, Anselm Levskaya, Sanjay Ghemawat, Sunipa Dev, Henryk Michalewski, Xavier Garcia, Vedant Misra, Kevin Robinson, Liam Fedus, Denny Zhou, Daphne Ippolito, David Luan, Hyeontaek Lim, Barret Zoph, Alexander Spiridonov, Ryan Sepassi, David Dohan, Shivani Agrawal, Mark Omernick, Andrew M. Dai, Thanumalayan Sankaranarayana Pillai, Marie Pellat, Aitor Lewkowycz, Erica Moreira, Rewon Child, Oleksandr Polozov, Katherine Lee, Zongwei Zhou, Xuezhi Wang, Brennan Saeta, Mark Diaz, Orhan Firat, Michele Catasta, Jason Wei, Kathy Meier-Hellstern, Douglas Eck, Jeff Dean, Slav Petrov, and Noah Fiedel. 2022. PaLM: Scaling Language Modeling with Pathways. *CoRR* abs/2204.02311 (2022). https://doi.org/10.48550/arXiv.2204.02311 arXiv:2204.02311

[15] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. 2021. Training Verifiers to Solve Math Word Problems. *CoRR* abs/2110.14168 (2021). arXiv:2110.14168 https://arxiv.org/abs/2110.14168

[16] Garett Cunningham, Razvan C. Bunescu, and David Juedes. 2023. Towards Autoformalization of Mathematics and Code Correctness: Experiments with Elementary Proofs. https://doi.org/10.48550/ARXIV.2301.02195

[17] Łukasz Czajka and Cezary Kaliszyk. 2018. Hammer for Coq: Automation for Dependent Type Theory. *Journal of Automated Reasoning* 61, 1-4 (2018), 423–453. https://doi.org/10.1007/s10817-018-9458-4

[18] Loris D'Antoni, Roopsha Samanta, and Rishabh Singh. 2016. Qlose: Program Repair with Quantitative Objectives. In *International Conference on Computer Aided Verification (CAV)*. Toronto, ON, Canada, 383–401.

[19] Angela Fan, Mike Lewis, and Yann Dauphin. 2018. Hierarchical Neural Story Generation. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, Melbourne, Australia, 889–898. https://doi.org/10.18653/v1/P18-1082

[20] Emily First and Yuriy Brun. 2022. Diversity-Driven Automated Formal Verification. In *International Conference on Software Engineering (ICSE)*. Pittsburgh, PA, 749–761. https://doi.org/10.1145/3510003.3510138

[21] Emily First and Yuriy Brun. 2022. Diversity-Driven Automated Formal Verification. In *ICSE* (22–27). 749–761. https://doi.org/10.1145/3510003.3510138

[22] Emily First, Yuriy Brun, and Arjun Guha. 2020. TacTok: Semantics-Aware Proof Synthesis. *PACMPL OOPSLA* 4 (November 2020), 231:1–231:31. https://doi.org/10.1145/3428299

[23] Thibault Gauthier, Cezary Kaliszyk, Josef Urban, Ramana Kumar, and Michael Norrish. 2021. TacticToe: Learning to Prove with Tactics. *J. Autom. Reason.* 65, 2 (feb 2021), 257–286. https://doi.org/10.1007/s10817-020-09580-x

[24] Sumit Gulwani, Ivan Radiček, and Florian Zuleger. 2018. Automated Clustering and Program Repair for Introductory Programming Assignments. In *PLDI*. 465–480. https://doi.org/10.1145/3192366.3192387

[25] Rahul Gupta, Soham Pal, Aditya Kanade, and Shirish K. Shevade. 2017. DeepFix: Fixing Common C Language Errors by Deep Learning. In *AAAI*.

[26] Christopher Hahn, Frederik Schmitt, Julia J. Tillman, Niklas Metzger, Julian Siber, and Bernd Finkbeiner. 2022. Formal Specifications from Natural Language. *CoRR* abs/2206.01962 (2022). https://doi.org/10.48550/arXiv.2206.01962

arXiv:2206.01962

[27] Jesse Michael Han, Jason Rute, Yuhuai Wu, Edward W. Ayers, and Stanislas Polu. 2022. Proof Artifact Co-Training for Theorem Proving with Language Models. In *The Tenth International Conference on Learning Representations, ICLR 2022, Virtual Event, April 25-29, 2022*. OpenReview.net. https://openreview.net/forum?id=rpxJc9j04U

[28] Mark Harman. 2007. The Current State and Future of Search Based Software Engineering. In *ACM/IEEE International Conference on Software Engineering (ICSE)*. 342–357. https://doi.org/10.1109/FOSE.2007.29

[29] Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. 2021. Measuring Mathematical Problem Solving With the MATH Dataset. *CoRR* abs/2103.03874 (2021). arXiv:2103.03874 https://arxiv.org/abs/2103.03874

[30] Jónathan Heras and Ekaterina Komendantskaya. 2014. ACL2(ml): Machine-learning for ACL2. *Electronic Proceedings in Theoretical Computer Science* 152 (04 2014). https://doi.org/10.4204/EPTCS.152.5

[31] Daniel Huang, Prafulla Dhariwal, Dawn Song, and Ilya Sutskever. 2019. GamePad: A Learning Environment for Theorem Proving. In *7th International Conference on Learning Representations, ICLR 2019, New Orleans, LA, USA, May 6-9, 2019*. OpenReview.net. https://openreview.net/forum?id=r1xwKoR9Y7

[32] Kush Jain, Karl Palmskog, Ahmet Celik, Emilio Jesús Gallego Arias, and Milos Gligoric. 2020. MCoq: Mutation Analysis for Coq Verification Projects. In *International Conference on Software Engineering Demonstrations Track (ICSE DEMO)*. Seoul, South Korea, 89–92. https://doi.org/10.1145/3377812.3382156

[33] Albert Jiang, Konrad Czechowski, Mateja Jamnik, Piotr Milos, Szymon Tworkowski, Wenda Li, and Yuhuai Tony Wu. 2022. Thor: Wielding Hammers to Integrate Language Models and Automated Theorem Provers. In *Neural Information Processing Systems (NeurIPS)*.

[34] Albert Qiaochu Jiang, Wenda Li, Jesse Michael Han, and Yuhuai Wu. 2021. LISA: Language models of ISAbelle proofs. In *Conference on Artificial Intelligence and Theorem Proving (AITP*. Aussois, France, 17.1–17.3.

[35] Albert Q. Jiang, Sean Welleck, Jin Peng Zhou, Wenda Li, Jiacheng Liu, Mateja Jamnik, Timothée Lacroix, Yuhuai Wu, and Guillaume Lample. 2022. Draft, Sketch, and Prove: Guiding Formal Theorem Provers with Informal Proofs. *CoRR* abs/2210.12283 (2022). https://doi.org/10.48550/arXiv.2210.12283 arXiv:2210.12283

[36] Jiajun Jiang, Yingfei Xiong, and Xin Xia. 2019. A manual inspection of Defects4J bugs and its implications for automatic program repair. *Science China Information Sciences* 62, 10 (2019), 200102.

[37] Jiajun Jiang, Yingfei Xiong, Hongyu Zhang, Qing Gao, and Xiangqun Chen. 2018. Shaping Program Repair Space with Existing Patches and Similar Code. In *ISSTA*. https://doi.org/10.1145/3213846.3213871

[38] Yalin Ke, Kathryn T. Stolee, Claire Le Goues, and Yuriy Brun. 2015. Repairing Programs with Semantic Code Search. In *ASE* (9–13). 295–306. https://doi.org/10.1109/ASE.2015.60

[39] Dongsun Kim, Jaechang Nam, Jaewoo Song, and Sunghun Kim. 2013. Automatic patch generation learned from human-written patches. In *ACM/IEEE International Conference on Software Engineering (ICSE)*. San Francisco, CA, USA, 802–811. http://dl.acm.org/citation.cfm?id=2486788.2486893

[40] Gerwin Klein, Kevin Elphinstone, Gernot Heiser, June Andronick, David Cock, Philip Derrin, Dhammika Elkaduwe, Kai Engelhardt, Rafal Kolanski, Michael Norrish, Thomas Sewell, Harvey Tuch, and Simon Winwood. 2009. SeL4: Formal Verification of an OS Kernel. In *Proceedings of the ACM SIGOPS 22nd Symposium on Operating Systems Principles* (Big Sky, Montana, USA) *(SOSP '09)*. Association for Computing Machinery, New York, NY, USA, 207–220. https://doi.org/10.1145/1629575.1629596

[41] Anil Koyuncu, Kui Liu, Tegawendé F Bissyandé, Dongsun Kim, Martin Monperrus, Jacques Klein, and Yves Le Traon. 2019. iFixR: Bug Report Driven Program Repair. In *ESEC/FSE*. 314–325. https://doi.org/10.1145/3338906.3338935

[42] Guillaume Lample, Marie-Anne Lachaux, Thibaut Lavril, Xavier Martinet, Amaury Hayat, Gabriel Ebner, Aurélien Rodriguez, and Timothée Lacroix. 2022. HyperTree Proof Search for Neural Theorem Proving. *CoRR* abs/2205.11491 (2022). https://doi.org/10.48550/arXiv.2205.11491 arXiv:2205.11491

[43] Leonidas Lampropoulos, Zoe Paraskevopoulou, and Benjamin C. Pierce. 2017. Generating Good Generators for Inductive Relations. *Proceedings of the ACM on Programming Languages (PACMPL)* 2, POPL (Dec. 2017), 45:1–45:30. https://doi.org/10.1145/3158133

[44] Xuan Bach D. Le, David Lo, and Claire Le Goues. 2016. History Driven Program Repair. In *Intl. Conf. on Software Analysis, Evolution, and Reengineering*, Vol. 1. 213–224. https://doi.org/10.1109/SANER.2016.76

[45] Claire Le Goues, ThanhVu Nguyen, Stephanie Forrest, and Westley Weimer. 2012. GenProg: A Generic Method for Automatic Software Repair. *IEEE Transactions on Software Engineering (TSE)* 38 (2012), 54–72. https://doi.org/10.1109/TSE.2011.104

[46] Claire Le Goues, Michael Pradel, and Abhik Roychoudhury. 2019. Automated Program Repair. *CACM* 62, 12 (Nov. 2019), 56–65. https://doi.org/10.1145/3318162

[47] Xavier Leroy. 2009. Formal Verification of a Realistic Compiler. *ACM* 52, 7 (July 2009), 107–115. https://doi.org/10.1145/1538788.1538814

[48] Aitor Lewkowycz, Anders Andreassen, David Dohan, Ethan Dyer, Henryk Michalewski, Vinay V. Ramasesh, Ambrose Slone, Cem Anil, Imanol Schlag, Theo Gutman-Solo, Yuhuai Wu, Behnam Neyshabur, Guy Gur-Ari, and Vedant Misra. 2022. Solving Quantitative Reasoning Problems with Language Models. *CoRR* abs/2206.14858 (2022). https://doi.org/10.48550/arXiv.2206.14858 arXiv:2206.14858

[49] Fan Long and Martin Rinard. 2016. Automatic Patch Generation by Learning Correct Code. In *ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*. St. Petersburg, FL, USA, 298–312. https://doi.org/10.1145/2837614.2837617

[50] Yiling Lou, Ali Ghanbari, Xia Li, Lingming Zhang, Haotian Zhang, Dan Hao, and Lu Zhang. 2020. Can Automated Program Repair Refine Fault Localization? A Unified Debugging Approach. In *ISSTA*. 75–87. https://doi.org/10.1145/3395363.3397351

[51] Paolo Masci and Aaron Dutle. 2022. Proof Mate: An Interactive Proof Helper for PVS (Tool Paper). In *NASA Formal Methods Symposium*. Springer, 809–815.

[52] Sergey Mechtaev, Manh-Dung Nguyen, Yannic Noller, Lars Grunske, and Abhik Roychoudhury. 2018. Semantic Program Repair Using a Reference Implementation. In *ICSE*. 129–139. https://doi.org/10.1145/3180155.3180247

[53] Manish Motwani and Yuriy Brun. 2023. Better Automatic Program Repair by Using Bug Reports and Tests Together. In *International Conference on Software Engineering (ICSE)* (14–20). Melbourne, Australia.

[54] Manish Motwani, Mauricio Soto, Yuriy Brun, René Just, and Claire Le Goues. 2022. Quality of Automated Program Repair on Real-World Defects. *TSE* 48, 2 (February 2022), 637–661. https://doi.org/10.1109/TSE.2020.2998785

[55] Yutaka Nagashima and Yilun He. 2018. PaMpeR: Proof Method Recommendation System for Isabelle/HOL. In *International Conference on Automated Software Engineering (ASE)*. Montpellier, France, 362–372. https://doi.org/10.1145/3238147.3238210

[56] Pengyu Nie, Karl Palmskog, Junyi Jessy Li, and Milos Gligoric. 2020. Deep Generation of Coq Lemma Names Using Elaborated Terms. In *International Joint Conference on Automated Reasoning (IJCAR)*. Paris, France, 97–118.

[57] Pengyu Nie, Karl Palmskog, Junyi Jessy Li, and Milos Gligoric. 2020. Learning to Format Coq Code Using Language Models. In *The Coq Workshop*. Aubervilliers, France.

[58] Pengyu Nie, Karl Palmskog, Junyi Jessy Li, and Milos Gligoric. 2021. Roosterize: Suggesting Lemma Names for Coq Verification Projects Using Deep Learning. In *International Conference on Software Engineering Demonstrations Track (ICSE DEMO)*. Madrid, Spain, 21–24. https://doi.org/10.1109/ICSE-Companion52605.2021.00026

[59] Kunihiro Noda, Yusuke Nemoto, Keisuke Hotta, Hideo Tanida, and Shinji Kikuchi. 2020. Experience Report: How Effective is Automated Program Repair for Industrial Software?. In *SANER*. 612–616.

[60] Kimia Noorbakhsh, Modar Sulaiman, Mahdi Sharifi, Kallol Roy, and Pooyan Jamshidi. 2021. Pretrained Language Models are Symbolic Mathematics Solvers too! *CoRR* abs/2110.03501 (2021). https://arxiv.org/abs/2110.03501

[61] Maxwell I. Nye, Anders Johan Andreassen, Guy Gur-Ari, Henryk Michalewski, Jacob Austin, David Bieber, David Dohan, Aitor Lewkowycz, Maarten Bosma, David Luan, Charles Sutton, and Augustus Odena. 2021. Show Your Work: Scratchpads for Intermediate Computation with Language Models. *CoRR* abs/2112.00114 (2021). arXiv:2112.00114 https://arxiv.org/abs/2112.00114

[62] Aditya Paliwal, Sarah M. Loos, Markus N. Rabe, Kshitij Bansal, and Christian Szegedy. 2020. Graph Representations for Higher-Order Logic and Theorem Proving. In *The Thirty-Fourth AAAI Conference on Artificial Intelligence, AAAI 2020, The Thirty-Second Innovative Applications of Artificial Intelligence Conference, IAAI 2020, The Tenth AAAI Symposium on Educational Advances in Artificial Intelligence, EAAI 2020, New York, NY, USA, February 7-12, 2020*. AAAI Press, 2967–2974.

[63] Karl Palmskog, Ahmet Celik, and Milos Gligoric. 2018. PiCoq: Parallel Regression Proving for Large-Scale Verification Projects. In *ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*. Amsterdam, Netherlands, 344–355. https://doi.org/10.1145/3213846.3213877

[64] Larry Paulson and Tobias Nipkow. 2023. The Sledgehammer: Let Automatic Theorem Provers write your Isabelle scripts! https://isabelle.in.tum.de/website-Isabelle2009-1/sledgehammer.html.

[65] Justyna Petke and Aymeric Blot. 2018. Refining Fitness Functions in Test-Based Program Repair. In *APR*. 13–14. https://doi.org/10.1145/3387940.3392180

[66] Stanislas Polu and Ilya Sutskever. 2020. Generative Language Modeling for Automated Theorem Proving. *CoRR* abs/2009.03393 (2020). arXiv:2009.03393 https://arxiv.org/abs/2009.03393

[67] Reiner Pope, Sholto Douglas, Aakanksha Chowdhery, Jacob Devlin, James Bradbury, Anselm Levskaya, Jonathan Heek, Kefan Xiao, Shivani Agrawal, and Jeff Dean. 2022. Efficiently Scaling Transformer Inference. https://doi.org/10.48550/ARXIV.2211.05102

[68] Zichao Qi, Fan Long, Sara Achour, and Martin Rinard. 2015. An Analysis of Patch Plausibility and Correctness for Generate-and-validate Patch Generation Systems. In *ISSTA*. 24–36. https://doi.org/10.1145/2771783.2771791